

# Inhalt

|                           |   |
|---------------------------|---|
| Methoden & Kapselung..... | 0 |
| › Lernziele.....          | 0 |
| › Einstieg.....           | 1 |
| › Kerninhalt.....         | 1 |
| › Aufgaben.....           | 6 |
| › KI-Prompts.....         | 8 |
| › Weiterführend.....      | 8 |
| › Reflexion.....          | 8 |
| › Quellen.....            | 9 |

## Methoden & Kapselung

### Arbeitsblatt G2 | Modul OOP - Herbst 2026

#### Kopfdaten

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| Thema           | G2 - Methoden & Kapselung                           |
| Voraussetzungen | G1 (Klassen & Eigenschaften)                        |
| Schwierigkeit   | ☆                                                   |
| Zeitaufwand     | 1-2 Doppelstunden                                   |
| Folgethemen     | E1, E2 (Bereich 2) sowie Bereich 6 (OOP für Profis) |

**Bevor du startest:** Fülle P1 (Vor dem Start) deines Themen-Planungsdokuments aus (Vorwissen, Ziele, Vorgehen). Es dauert nur wenige Minuten und macht dein Lernen gezielter.



#### Lernziele

Was du nach diesem Arbeitsblatt kannst, siehst du in der folgenden Liste. Sie hilft dir, deinen Lernfortschritt selbst einzuschätzen.

## Lernziele

Nach diesem Arbeitsblatt kannst du:

- › erklären, was eine Methode ist, und Methoden mit Parametern und Rückgabewert in einer Klasse schreiben und aufrufen;
- › den Aufbau einer Methode (Zugriffstyp, Rückgabebetyp, Name, Parameter, `return`) benennen und anwenden;
- › Konstruktoren schreiben, ihren Zweck erklären und Konstruktoren überladen;
- › das Prinzip der **Kapselung** (engl. encapsulation / information hiding) erklären und begründen, warum es eine grundlegende OOP-Kompetenz ist;
- › eine Klasse so gestalten, dass interne Daten geschützt und nur über eine kontrollierte Schnittstelle (Methoden/Eigenschaften) zugänglich sind.

Vertiefung / PRO (freiwillig, nicht prüfungsrelevant):

- › statische Methoden kennen und gezielt einsetzen;

- › erklären, wie sich Wert- und Referenztypen bei der Übergabe als Parameter verhalten (byValue / byReference), und was ein Seiteneffekt ist.

**Anschluss an G1:** In G1 hast du Klassen als Datenspeicher (Felder, Eigenschaften) gebaut. Jetzt erhalten die Klassen Verhalten (Methoden) - und du lernst, wie du Daten und Verhalten sauber zu einer geschützten Einheit zusammenfügst. Damit werden deine Klassen «komplett».

# Einstieg

Eine Klasse Konto mit einem öffentlichen Feld `Saldo` lässt sich von aussen beliebig verändern: `konto.Saldo = -5000`; ist möglich, obwohl das fachlich Unsinn sein kann. Wer schützt die Daten davor?

**Denkanstoss (zuerst selbst überlegen):** Notiere kurz, wie du verhindern würdest, dass jemand von aussen einen ungültigen Kontostand setzt. Welche Regeln würdest du wo unterbringen? Vergleiche deine Idee am Ende mit dem Abschnitt «Kapselung».

Die Antwort der OOP: Daten werden **gekapselt** und nur über klar definierte Methoden bzw. Eigenschaften verändert. Dorthin legst du die Regeln. Doch zuerst zu den Methoden selbst.

# Kerninhalt

## Methoden

Eine **Methode** ist ein benannter Block von Anweisungen, der eine bestimmte Aufgabe erfüllt. Methoden gehören zu einer Klasse und definieren das Verhalten von Objekten. Aus der prozeduralen Programmierung kennst du sie als Funktionen; in der OOP gehören sie zu einer Klasse und können auf deren Daten (Felder, Eigenschaften) zugreifen.

Schema einer Methode:

```
Zugriffstyp Rückgabetyp Methodenname(Parameterliste)
{
    // Anweisungen
    return Rückgabewert;
}
```

Hauptkomponenten:

| Komponente          | Beschreibung                                                          |
|---------------------|-----------------------------------------------------------------------|
| Zugriffstyp         | <code>public</code> , <code>private</code> , ... wie bei Feldern      |
| Rückgabetyp         | Datentyp des Rückgabewerts oder <code>void</code> (kein Rückgabewert) |
| Methodenname        | Name der Methode (PascalCase, Clean Code)                             |
| Parameterliste      | Übergabewerte (optional)                                              |
| <code>return</code> | Rückgabe des Wertes (entfällt bei <code>void</code> )                 |

Beispiel einer Methode in der Klasse Buch:

```
internal class Buch
{
    public string Titel;
    public string Autor;
    public int Seiten;

    public void AusgabeInfos()
    {
        Console.WriteLine($"{Titel} von {Autor} ({Seiten} Seiten)");
    }
}
```

Aufruf aus einem Objekt:

```
Buch buch = new Buch { Titel = "C#", Autor = "Max Mustermann",
Seiten = 300 };
buch.AusgabeInfos();
```

**Übung:** Schreibe eine Methode, welche die Anzahl Wörter im Titel eines Buchs zählt und zurückgibt. Tipp: `Split(' ')` zerlegt einen String am Leerzeichen in ein Array.

Mögliche Lösung:

```
internal class Buch
{
    public string Titel;
    public string Autor;
    public int Seiten;

    public int AnzahlWoerterImTitel()
    {
        return Titel.Split(' ').Length;
    }
}
```

## Konstrukturen

Ein **Konstruktor** ist eine spezielle Methode, die beim Erstellen (Instanzieren) eines Objekts automatisch aufgerufen wird. Er initialisiert das Objekt, setzt also z.B. Anfangswerte.

Eigenschaften eines Konstruktors:

- › gleicher Name wie die Klasse;
- › kein Rückgabetyt (auch nicht `void`);
- › wird automatisch beim `new` aufgerufen.

```
internal class Buch
{
    public string Titel;
    public string Autor;

    public Buch(string titel, string autor)
    {
        Titel = titel;
        Autor = autor;
    }
}
```

Anwendung:

```
Buch buch = new Buch("C#", "Max Mustermann");
```

Wird kein Konstruktor definiert, stellt C# automatisch einen leeren Standardkonstruktor bereit. Sobald du einen eigenen Konstruktor definierst, entfällt dieser - bei Bedarf musst du ihn manuell ergänzen:

```
public Buch() { }
```

**Überladung von Konstruktoren:** Es können mehrere Konstruktoren mit unterschiedlichen Parameterlisten existieren. So lässt sich ein Objekt auf verschiedene Arten erstellen:

```
internal class Buch
{
    public string Titel;
    public string Autor;
    public int Seiten;

    public Buch(string titel, string autor)
    {
        Titel = titel;
        Autor = autor;
    }

    public Buch(string titel, string autor, int seiten)
    {
        Titel = titel;
        Autor = autor;
        Seiten = seiten;
    }
}
```

Das nennt man **Überladung** (engl. overloading).

## Kapselung – der zentrale Grundsatz

Die **Kapselung** (engl. encapsulation, eng verwandt mit dem information hiding) ist eines der grundlegenden Prinzipien der objektorientierten Programmierung. Die Idee: Ein Objekt verbirgt seine internen Daten und gibt sie nur über eine klar definierte Schnittstelle (öffentliche Methoden bzw. Eigenschaften) preis.

Diese Idee ist älter als die OOP selbst. David Parnas formulierte 1972 das Prinzip des *information hiding*: Ein Modul soll seine Entwurfsentscheidungen - besonders jene, die sich ändern könnten - vor anderen Modulen verbergen und nur eine stabile Schnittstelle anbieten (Parnas, 1972). Übertragen auf Klassen heisst das: Die interne Datenhaltung ist ein «Geheimnis» der Klasse; von aussen sichtbar ist nur, *was* die Klasse anbietet, nicht *wie* sie es intern löst. Alan Snyder zeigte 1986, dass genau diese saubere Trennung von externer Schnittstelle und interner Repräsentation entscheidend dafür ist, dass sich die Implementierung einer Klasse ändern lässt, ohne den Code ihrer Nutzer zu brechen (Snyder, 1986). Bertrand Meyer ordnet die Kapselung als Werkzeug der Software-Qualität ein: Sie senkt die Kopplung zwischen Bausteinen und erhöht damit Wartbarkeit und Wiederverwendbarkeit (Meyer, 1997).

Konkret bedeutet Kapselung in C#:

- › Felder werden `private` gehalten (das «Geheimnis» der Klasse);
- › der Zugriff erfolgt über `public` Methoden oder Eigenschaften (die «Schnittstelle»);
- › in dieser Schnittstelle liegen die Regeln: Gültigkeitsprüfungen, Berechnungen, Folgeaktionen.

Beispiel: eine Klasse `Konto`, die einen negativen Saldo durch unkontrollierten Zugriff verhindert.

```
internal class Konto
{
    private decimal saldo;           // privates Feld: das
    Geheimnis

    public decimal Saldo             // öffentliche
    Schnittstelle (nur lesen)
    {
        get { return saldo; }
    }

    public void Einzahlen(decimal betrag) // kontrollierte
    Veränderung
    {
        if (betrag <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(betrag), "Der Betrag muss positiv sein.");
        }
        saldo += betrag;
    }

    public void Abheben(decimal betrag)
    {
        if (betrag <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(betrag), "Der Betrag muss positiv sein.");
        }
        if (betrag > saldo)
        {
            throw new InvalidOperationException("Kontostand reicht
nicht aus.");
        }
        saldo -= betrag;
    }
}
```

Der Saldo lässt sich nun nicht mehr von aussen frei setzen - nur über Einzahlen und Abheben, und dort gelten die Regeln. Das ist der praktische Kern von Kapselung: **Die Klasse schützt ihre eigene Konsistenz.**

Warum das eine essenzielle Kompetenz ist:

- › **Sicherheit gegen ungültige Zustände:** Ein Objekt kann nicht in einen unsinnigen Zustand gebracht werden, weil alle Änderungen durch geprüfte Methoden laufen.
- › **Änderbarkeit ohne Bruch:** Die interne Umsetzung darf sich ändern (z.B. anderes Feld, andere Berechnung), solange die Schnittstelle gleich bleibt - der nutzende Code merkt nichts (Snyder, 1986).
- › **Weniger Kopplung, bessere Wartbarkeit:** Nutzer hängen nur von der Schnittstelle ab, nicht von Interna (Meyer, 1997).

**Brücke zu G1:** Die Werkzeuge der Kapselung - private Felder, Eigenschaften mit Getter/Setter, Validierung im Setter - hast du in G1 bereits kennengelernt. Hier bekommen sie ihren Namen

und ihren Zweck. Auch ein **Trigger** (eine Folgeaktion im Setter, z.B. ein «letzte Änderung»-Zeitstempel oder ein Log-Eintrag) ist ein Beispiel kontrollierten Zugriffs.

### Beispiel mit Lücken (Einstiegshilfe)

Lernpfad B: Ergänze die mit \_\_\_\_ markierten Stellen selbst. Die vollständige Version steht oben im Kerninhalt als Kontrolle.

```
internal class Buch
{
    public string Titel;
    public string Autor;

    // Methode ergänzen: Rückgabebetyp?
    public ____ AusgabeInfos()
    {
        Console.WriteLine($"{Titel} von {____}");
    }

    // Konstruktor ergänzen
    public Buch(string titel, string autor)
    {
        ____ = titel;
        Autor = ____;
    }
}
```

## Aufgaben



### Arbeitsanweisung

Bearbeite die Basisaufgaben verbindlich. Vertiefung und Challenge sind für mehr Tiefe bzw. für Schnelle gedacht. KI darfst du nutzen - zum Erklären und Prüfen, nicht zum blinden Kopieren.

## Basis (Pflicht)

**Aufgabe 1 - Methoden in Klassen.** Erweitere die Klassen deines eigenen Fallbeispiels (aus G1) um mindestens drei sinnvolle Methoden. Mindestens eine davon hat einen Parameter und einen Rückgabewert.

**Aufgabe 2 - Konstruktoren mit Überladung.** Ergänze dein Fallbeispiel um mindestens zwei Konstruktoren (Überladung) und erstelle Objekte mit den verschiedenen Konstruktoren.

**Aufgabe 3 - Eine 1-m-Beziehung kapseln.** Erweitere die Klasse Konto so, dass sie ihre Transaktionen speichert - eine 1-m-Beziehung: ein Konto hat viele Transaktionen.

- › Lege eine kleine Klasse Transaktion an (z.B. Betrag und Datum).
- › Konto erhält eine private Liste von Transaktionen. Initialisiere sie im Konstruktor, damit die Klasse von Anfang an funktioniert (keine leere Referenz).
- › Biete nach aussen nur ein sauberes Interface: Einzahlen/Abheben legen automatisch eine Transaktion an; eine Methode gibt die Transaktionen (oder ihre Anzahl) zurück, ohne dass der Nutzer die interne Liste direkt verändern kann.

- › Halte fest: Der Nutzer muss nicht wissen, ob du intern Liste, Array, Stack oder Queue verwendest - das Interface bleibt gleich. Die Sammlungstypen selbst vertiefst du in X4.

Wende es danach auf ein eigenes Beispiel an (z.B. Tier - Fütterungen).

## Vertiefung

**Aufgabe 4 - Klasse Konto kapseln.** Baue die Klasse Konto so, dass der Saldo nur über Einzahlen und Abheben verändert werden kann und ungültige Beträge abgewiesen werden. Teste sie mit gültigen und ungültigen Werten und beobachte das Verhalten.

**Aufgabe 5 - Kapselung im eigenen Fallbeispiel.** Wähle eine Klasse deines Fallbeispiels und kapsle sie konsequent: private Felder, öffentliche Schnittstelle, mindestens eine Gültigkeitsprüfung und eine Folgeaktion (Trigger). Begründe in ein bis zwei Sätzen, welches «Geheimnis» die Klasse schützt und warum.

## Challenge (PRO, nicht prüfungsrelevant)

**Aufgabe 6 - statische Methoden.** Erstelle eine Hilfsklasse mit mindestens zwei statischen Methoden (z.B. Berechnungen) und rufe sie ohne Objekt direkt über die Klasse auf.

```
public static int Addiere(int a, int b)
{
    return a + b;
}

// Aufruf:
int summe = Rechner.Addiere(3, 4);
```

**Aufgabe 7 - byValue / byReference bei Parametern.** Sage zuerst die Ausgabe voraus, teste sie dann, und erkläre den Unterschied:

```
static void Adding1(int a)           // Werttyp
{
    a = a + 1;
}

static void RenameBuch(Buch buch)    // Referenztyp
{
    buch.Titel = "Neuer Titel";
}

int a = 5;
Adding1(a);
Console.WriteLine(a);                // Vorhersage? ____

Buch buch = new Buch { Titel = "C#" };
RenameBuch(buch);
Console.WriteLine(buch.Titel);       // Vorhersage? ____
```

Erkläre, warum a unverändert bleibt, der Buchtitel aber geändert wird. Recherchiere zusätzlich den Begriff **Seiteneffekt**: Warum gilt das Verändern eines übergebenen Objekts ohne Rückgabewert als schlechter Stil? Optional: das Schlüsselwort `ref` und seine Wirkung.

# KI-Prompts

KI ist in diesem Modul ein Pflicht-Werkzeug. Nutze diese Start-Prompts zum Erkunden, Erklären und Prüfen - und beurteile die Antworten kritisch:

- › **Erklären:** «Erkläre mir das Prinzip der Kapselung in C# an einem Alltagsbeispiel. Warum macht man Felder `private` und bietet öffentliche Methoden an?»
- › **Erkunden:** «Was ist der Unterschied zwischen einem Konstruktor und einer normalen Methode in C#? Wann brauche ich Konstruktor-Überladung?»
- › **Prüfen (Gegencheck):** «Hier ist meine Klasse Konto: [Code einfügen]. Kann ihr Zustand von aussen in einen ungültigen Saldo gebracht werden? Wo ist die Kapselung lückenhaft?»
- › **Vertiefung:** «Erkläre an einem Beispiel den Unterschied zwischen der Übergabe eines Werttyps und eines Referenztyps an eine Methode. Was ist ein Seiteneffekt?»

**Wichtig:** Halte deine KI-Nutzung im KI-Protokoll (Planungsblatt) fest: Prompt, was war hilfreich, was war falsch oder irreführend?

## Weiterführende Informationen

Die folgenden Links vertiefen das Thema. Sie sind freiwillig, aber für deinen Leistungsnachweis nützlich.

## Weiterführend

- › Datenkapselung - Grundprinzip (Wikipedia):  
[https://de.wikipedia.org/wiki/Datenkapselung\\_\(Programmierung\)](https://de.wikipedia.org/wiki/Datenkapselung_(Programmierung))
- › Eigenschaften und Kapselung (Microsoft Learn):  
<https://learn.microsoft.com/de-de/dotnet/csharp/properties>
- › Kapselung mit Eigenschaften - mit Video (programmieren-starten.de):  
<https://programmieren-starten.de/blog/csharp-eigenschaften/>
- › Methoden (Microsoft Learn):  
<https://learn.microsoft.com/de-de/dotnet/csharp/programming-guide/classes-and-structs/methods>
- › Konstruktoren (Microsoft Learn):  
<https://learn.microsoft.com/de-de/dotnet/csharp/programming-guide/classes-and-structs/constructors>
- › `static` (Microsoft Learn): <https://learn.microsoft.com/de-de/dotnet/csharp/language-reference/keywords/static>
- › Werttyp vs. Referenztyp (für Neugierige): <https://ericlippert.com/2015/05/18/when-everything-you-know-is-wrong-part-one/>

## Reflexion

Fülle P3 (Abschluss) deines Themen-Planungsdokuments aus. Kurz zusammengefasst:

Was kann ich jetzt, was ich vorher nicht konnte?

Warum ist Kapselung wichtig - in eigenen Worten?

# Quellen

- › Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053-1058.
- › Snyder, A. (1986). Encapsulation and inheritance in object-oriented programming languages. *OOPSLA '86, ACM SIGPLAN Notices*, 21(11), 38-45.
- › Meyer, B. (1997). *Object-Oriented Software Construction* (2. Aufl.). Prentice Hall.
- › Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. (Namens- und Gestaltungsregeln, «Clean Code»)